

Relazione — Progetto Connections (Lab III, UniPi)

Nota di lavoro: bozza da sintetizzare fino al limite di **5 pagine PDF** per la consegna finale. Contiene le 4 sezioni + manuale d'uso richiesti dalla specifica (sezione 4).

1. Scelte di interpretazione personale

(Punti del progetto lasciati alla libera interpretazione, con relativa motivazione)

- **Separazione ortogonale `MoveOutcome` / `GameOutcome`:** l'esito della singola proposta (`CORRECT`/`WRONG`/`MALFORMED`/`ALREADY_COMPLETED`) e l'esito finale della partita per il giocatore (`WON`/`LOST_BY_MISTAKES`/`DID_NOT_FINISH`) sono modellati come due campi distinti in `ProposalResult`, non un unico enum con tutte le combinazioni esplicite. Una singola mossa può essere sia `CORRECT` sia concludere contestualmente la partita (es. terzo gruppo trovato) — un enum piatto non potrebbe rappresentare entrambe le informazioni senza perderne una.
- **CQS (Command-Query Separation)** applicato rigorosamente in `getGameInfoForPlayer`/`getGameStats`: le query di sola lettura non usano `computeIfAbsent` e non creano mai una entry in `activePlayerStates` come effetto collaterale. L'inizializzazione dello stato di un giocatore avviene **solo** alla prima `submitProposal`.
- **`DID_NOT_FINISH` come convenzione implicita, non campo persistente:** `PlayerGameState.getOutcome()` è derivato da `correctGroups/mistakes`. Un `outcome == null` letto da un `PlayerGameState` dentro un `GameRecord` concluso è interpretato per convenzione come DNF (single source of truth, nessun campo ridondante).
- **Aggiornamento tempestivo delle statistiche utente:** `UserStats.recordGameResult(...)` viene invocato immediatamente in `submitProposal` non appena l'esito di un giocatore diventa definitivo, non solo alla rotazione globale del round. `rotateGame()` si occupa esclusivamente dei giocatori rimasti con `outcome == null` allo scadere del tempo. Scoperto e corretto grazie a un test end-to-end reale (senza questa correzione un giocatore doveva attendere la fine dell'intero round, anche dopo aver già vinto/perso, per vedere aggiornate le proprie statistiche e la classifica).
- **`MoveOutcome.MALFORMED` → `ResponseCode.MALFORMED_PROPOSAL`** (errore di protocollo, non `SUCCESS` con payload): scelta guidata dal testo esplicito della specifica ("notificate come errori al giocatore").
- **`ALREADY_COMPLETED` → `ResponseCode.BAD_REQUEST`:** tentativo di proposta su partita già conclusa per il giocatore, senza mutare stato né consumare tentativi.
- **Riuso del campo `entries` di `LeaderboardPayload`** sia per classifica generale/top-K sia per un singolo `playerName` (lista a un elemento), evitando di estendere un DTO di protocollo già definito.
- **Scheduling:** `scheduleWithFixedDelay` per `PersistenceManager` (evita sovrapposizioni di I/O lento) vs. `scheduleAtFixedRate` per `GameManager.rotateGame()` (operazione rapida in memoria; la durata del round non deve subire deriva temporale cumulativa).
- **Notifica UDP di fine round (Macro-4):** inviata **solo** alla scadenza globale del timer (`rotateGame()`), non alla vittoria/sconfitta anticipata di un singolo giocatore — quest'ultima è già comunicata in modo sincrono via TCP nella risposta a `submitProposal`. Motivazione: notificare via UDP anche gli altri partecipanti ancora attivi al momento della vittoria/sconfitta di uno di loro rivelerebbe prematuramente le soluzioni a chi sta ancora giocando. Il contenuto riusa `getGameInfoForPlayer` (personalizzato, soluzioni rivelate) + `getGameStats` (aggregato, comune), incapsulati in

`GameFinishedNotificationPayload`, inviati in unicast a ciascun partecipante che ha registrato un endpoint UDP al login.

- **Endpoint UDP comunicato al login:** la porta UDP locale del client (`DatagramSocket(0)`, porta effimera per evitare conflitti tra istanze multiple sulla stessa macchina) è inviata come campo opzionale `udpPort` nel messaggio di login, sfruttando l'estensibilità del protocollo JSON prevista dalla specifica (sezione 5). `SessionManager` è stato ridisegnato da `Set<String>` a `ConcurrentHashMap<String, UserSession>` (record con l'`InetSocketAddress` associato) per evitare due strutture dati parallele da tenere manualmente allineate.

2. Schema dei thread

Lato server

Thread	Ruolo
Main (accept loop)	<code>ServerSocket.accept()</code> bloccante, delega ogni connessione al thread pool
Worker pool (<code>newCachedThreadPool</code>)	Un thread per connessione client attiva, esegue <code>ClientHandler</code>
Timer <code>GameManager</code> (<code>ScheduledExecutorService</code> mono-thread)	Esegue <code>rotateGame()</code> a cadenza fissa (<code>scheduleAtFixedRate</code>), incluso l'invio delle notifiche UDP di fine round
Timer <code>PersistenceManager</code> (<code>ScheduledExecutorService</code> mono-thread)	Esegue <code>saveAll()</code> periodico (<code>scheduleWithFixedDelay</code>)
Invio UDP	Non un thread dedicato: l'invio (<code>UdpNotifier.sendNotification</code>) avviene sincronamente dentro il thread del timer di <code>GameManager</code> , dato che <code>send()</code> su <code>DatagramSocket</code> non è un'operazione bloccante significativa

Lato client

Thread	Ruolo
Main	Loop CLI, invio richieste TCP sincrone (<code>SocketChannel</code> bloccante)
<code>UDP-Notification-Worker</code> (avviato dopo login riuscito)	Ciclo bloccante su <code>DatagramSocket.receive(...)</code> , stampa asincrona delle notifiche di fine round senza bloccare il prompt; terminato al logout/uscita tramite chiusura del socket (<code>SocketException</code> intenzionale distinta da errori reali)

3. Strutture dati utilizzate

Lato server

- **SessionManager**: `ConcurrentHashMap<String, UserSession>` — unica fonte di verità per "chi è loggato" + relativo endpoint UDP (record `UserSession(InetSocketAddress udpEndpoint)`, campo interno nullable per client senza UDP).
- **UserRepository**: `ConcurrentHashMap<String, User>` — utenti persistiti in JSON.
- **GameRepository**: `ConcurrentHashMap<Integer, GameRecord>` — storico partite concluse.
- **GameManager**:
 - `activeGame` (campo semplice, protetto da monitor intrinseco `synchronized`)
 - `activePlayerStates`: `ConcurrentHashMap<String, PlayerGameState>` — stato mutabile dei giocatori nella partita attiva
 - `templates`: `Map<Integer, GameTemplate>` immutabile (`Collections.unmodifiableMap`), caricata una sola volta all'avvio
- **GameRecord**: snapshot con `allGroups` e `playerStates`, immutabile a livello di costruttore (nota: Gson bypassa questa garanzia in deserializzazione via reflection, popolando i campi direttamente).
- **UdpNotifier**: un unico `DatagramSocket` condiviso, bind sulla porta UDP configurata, aperto per tutta la vita del server.

Lato client

- **UdpNotificationListener**: possiede il proprio `DatagramSocket` (porta effimera), aperto prima del login e riaperto per tutta la sessione autenticata.
- Nessuna struttura dati concorrente lato client: un solo thread di ascolto UDP in aggiunta al thread principale, nessuno stato condiviso mutabile tra i due oltre alla stampa su stdout (nessuna sincronizzazione esplicita necessaria, l'unica risorsa condivisa — la console — è usata in sola scrittura senza necessità di atomicità tra le righe).

4. Primitive di sincronizzazione

- **GameManager**: tutti i metodi che leggono/modificano `activeGame/activePlayerStates/currentGameId` sono `synchronized` sullo stesso monitor intrinseco (`this`). La rientranza dei lock Java permette a `rotateGame()` di chiamare `startNewActiveGame()` senza deadlock.
 - **lifecycleLock dedicato**: `start()/stop()` del timer interno di **GameManager** usano un monitor **separato** da quello di dominio, per evitare che lo shutdown hook resti bloccato in attesa che una rotazione in corso (che detiene il lock di dominio) termini — rischio di stallo diagnosticato e risolto disaccoppiando i due monitor.
 - **UserStats**: tutti i metodi (getter e `recordGameResult`) sono `synchronized` sullo stesso oggetto istanza, per garantire atomicità e visibilità tra il thread di rotazione di **GameManager** (scrittore) e i thread **ClientHandler** (lettori, per `requestPlayerStats/requestLeaderboard`).
 - **SessionManager**: `ConcurrentHashMap` + operazione atomica `putIfAbsent` per prevenire race condition di tipo check-then-act sul doppio login, senza necessità di un blocco `synchronized` a grana grossa.
 - **Persistenza**: **UserRepository/GameRepository** con metodi `synchronized` per l'accesso alla mappa interna, condivisi tra i thread **ClientHandler** e il thread periodico di **PersistenceManager**.
 - **UdpNotifier**: nessuna sincronizzazione esplicita necessaria — `DatagramSocket.send()` è già thread-safe per chiamate concorrenti; l'invio avviene comunque sempre dallo stesso thread (quello del timer di **GameManager**), quindi la questione è più teorica che pratica in questo progetto.
-

5. Manuale di istruzioni

- Requisiti: JDK 17+, libreria Gson ([lib/gson-2.10.1.jar](#)).
- Compilazione da riga di comando:

```
javac -cp "lib/*:src" -d bin $(find src -name "*.java")
```

- Avvio server: `java -cp "bin:lib/*" server.ServerMain [percorso-config-opzionale]`
- Avvio client: `java -cp "bin:lib/*" client.ClientMain [percorso-config-opzionale]`
- File di configurazione:
 - `config/server.properties`:
 - `server.host` : Indirizzo IP o hostname su cui il server apre il socket di ascolto TCP (es. `127.0.0.1` o `localhost`).
 - `server.tcp.port` : Porta su cui il `ServerSocket` accetta le connessioni TCP persistenti dei client.
 - `server.udp.port` : Porta UDP locale impiegata dal server per inviare le notifiche asincrone di fine round ai client registrati.
 - `persistence.users.path` : Percorso relativo del file JSON impiegato per la memorizzazione permanente degli utenti (credenziali e statistiche storiche).
 - `persistence.games.path` : Percorso relativo del file JSON per la memorizzazione permanente dello storico dei record delle partite concluse.
 - `game.words.path` : Percorso relativo al dataset JSON contenente l'elenco dei template di gioco e anche delle soluzioni dei vari gruppi di parole.
 - `persistence.flush.interval.minutes` : intervallo temporale in minuti tra due salvataggi periodici automatici consecutivi su disco dello stato degli utenti e delle partite.
 - `game.duration.minutes` : Durata in minuti di ogni partita globale; al termine del tempo, il server conclude il round e notifica i partecipanti tramite messaggio UDP.
 - `config/client.properties`:
 - `server.host`: Indirizzo IP o hostname del server Connections a cui connettersi via TCP(es. `127.0.0.1`).
 - `server.port`: Porta TCP del server su cui inoltrare la richiesta di connessione, deve corrispondere a `server.tcp.port` del file `server.properties`.
- **Comandi CLI lato client** : L'interfaccia opera mediante 2 menù numerati in base allo stato della sessione:
 - Menù Non Autenticato:
 - **1. Registrazione** : Richiede `username` e `password` per creare un nuovo account (`register`).
 - **2. Login** : Alloca il socket UDP locale, invia credenziali e porta UDP (`login`), avvia il thread ricevitore e mostra le parole del round attivo con errori e punteggio parziale.
 - **0. Esci** : Chiude i canali di comunicazione e termina l'applicazione.
 - Menù Autenticato:

- **1. Invia Proposta:** Riceve 4 parole separate da virgola (`submitProposal`) e visualizza l'esito (`CORRECT`/`WRONG`/`MALFORMED`), il gruppo individuato o gli errori residui.
 - **2. Richiedi Stato Partita:** Mostra lo stato di una partita specificando il `gameId` (indicare `0` per la partita corrente) (`requestGameInfo`).
 - **3. Logout:** Notifica il server (`logout`), arresta il thread di ascolto UDP, chiude il socket e torna al menu pre-login.
 - **4. Aggiorna Credenziali:** Modifica nome utente, password o entrambi (`updateCredentials`).
 - **5. Statistiche Partita:** Mostra le metriche aggregate di un round specificando il `gameId` (`0` per il corrente) (`requestGameStats`).
 - **6. Classifica:** Recupera l'intera graduatoria, i top-K giocatori o il punteggio di un utente specifico (`requestLeaderboard`).
 - **7. Statistiche Personali:** Visualizza la carriera del giocatore (win/loss rate, streak e istogramma degli errori) (`requestPlayerStats`).
- Packaging ed esecuzione dei file JAR: I file JAR sia per client che per server vengono generati includendo i metadati di avvio (`Main-Class`) e anche il percorso relativo per la libreria Gson (`Class-Path`) nel manifest, andando a separare i package di competenza (`client` e `server`) e invece condividendo il package che hanno in comune `common`. Tutte le istruzioni vanno eseguite dalla radice del progetto.

- Compilazione dei sorgenti Java:

```
mkdir -p bin
javac -cp "lib/gson-2.10.1.jar:src" -d bin $(find src -name "*.java")
```

- Creazione degli archivi JAR eseguibili

```
printf "Main-Class: server.ServerMain\nClass-Path: lib/gson-2.10.1.jar\n\n" > manifest-server.txt
printf "Main-Class: client.ClientMain\nClass-Path: lib/gson-2.10.1.jar\n\n" > manifest-client.txt

jar cfm Server.jar manifest-server.txt -C bin server -C bin common
jar cfm Client.jar manifest-client.txt -C bin client -C bin common

rm manifest-server.txt manifest-client.txt
```

- Esecuzione delle applicazioni: I file JAR devono essere eseguiti dalla directory radice, per una corretta risoluzione dei percorsi relativi verso `config/` e `data/`

```
# Avvio del Server (Terminale 1)
java -jar Server.jar
```

```
# Avvio del Client (Terminale 2)
java -jar Client.jar
```

Checklist di completamento (uso interno, non va nella relazione finale)

- ☒ Macro-0, Macro-1, Macro-2, Phase 5.1
- ☒ Macro-3 (concorrenza base)
- ☒ Macro-6 (ciclo di vita partita, **GameManager** completo)
- ☒ **requestGameStats**, **requestLeaderboard**, **requestPlayerStats**
- ☒ Macro-4 (UDP server) — completata e testata end-to-end (2 client reali, stesso round, stessa notifica)
- ☒ Phase 5.2 (UDP client) — completata e testata end-to-end
- ☒ Completare manuale d'uso (**client.properties**, screenshot/elenco comandi)
- [x] Packaging JAR
- ☐ Rilettura finale sezione 5 della specifica (formati messaggi) uno per uno
- ☐ Sintesi finale entro il limite di 5 pagine PDF